BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Practical No: 6

Division: C Batch: 2

Roll No: 3090341 PRN No: 12410595

Name of Student: Pranav Khatod

Subject Name: Deep Learning

Problem Statement:

Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Methodology:

The proposed system follows these steps:
Dataset Collection → Image Preprocessing → Train/Validation Split → CNN Model → Model Training → Prediction → Performance Evaluation

A. Image Preprocessing

Images have different dimensions, so they are resized to 128 × 128 pixels. Pixel values are normalized from the range 0–255 to 0–1 using a Rescaling layer.

B. CNN Architecture

The CNN consists of the following layers:

1. **Rescaling Layer**
   Normalises pixel values to improve model training.
2. **Convolutional Layers**
   Three convolution layers with 32, 64, and 128 filters extract important features such as edges, textures, spots, and disease patterns.

3. **Max Pooling Layers**
   Reduce the spatial dimensions of feature maps while retaining important information.
4. **Flatten Layer**
   Converts the extracted feature maps into a one-dimensional vector.
5. **Dense Layer**
   A fully connected layer with 128 neurons learns complex relationships between extracted features.
6. **Dropout Layer**
   Uses a dropout rate of 0.5 to reduce overfitting.
7. **Output Layer**
   Uses the Softmax activation function to classify the image into one of the disease categories.

Dataset:

The Corn or Maize Leaf Disease Dataset from Kaggle is used. The images are organized into different classes representing healthy and diseased maize leaves.

The dataset is divided into:

- 80% Training data – used to train the CNN.
- 20% Validation data – used to evaluate the model during training.

Each image is resized to 128 × 128 pixels and processed in batches of 32 images.

Model Compilation:

The model uses:

- Optimizer: Adam
- Learning Rate: 0.0001
- Loss Function: Sparse Categorical Cross-Entropy
- Evaluation Metric: Accuracy
- Number of Epochs: 10
- Batch Size: 32

The Adam optimizer updates the model weights efficiently during training, while sparse categorical cross-entropy measures the difference between actual and predicted classes.

Model Training:

The CNN is trained using the training dataset for 10 epochs. During each epoch, the model learns disease-related features from the training images.

The validation dataset is used to monitor the model's performance on unseen images. Training and validation accuracy and loss are recorded for every epoch.

Performance Evaluation:

The trained model is evaluated using the following techniques:

Accuracy

Accuracy represents the percentage of correctly classified images.

Accuracy = Correct Predictions / Total Prediction

Confusion Matrix:

A confusion matrix shows the number of correctly and incorrectly classified images for each disease class. It helps identify which diseases the model may confuse with each other.

Precision

Precision indicates how many images predicted as a particular disease actually belong to that disease.

Recall

Recall indicates how many actual images of a particular disease were correctly identified.

F1-Score

F1-score is the harmonic mean of precision and recall and provides a balanced measure of classification performance.

Results and Analysis:

The training and validation accuracy graphs are used to observe the learning behavior of the model. Similarly, training and validation loss graphs show how the prediction error changes during training.

The confusion matrix provides a class-wise evaluation of the CNN. The classification report provides precision, recall, F1-score, and support for every disease class.

Result: The CNN is able to automatically learn visual characteristics of corn/maize leaf diseases and classify leaf images into their respective disease categories.

Code :

```
import kagglehub

path = kagglehub.dataset_download("smaranjitghose/corn-or-maize-leaf-disease-dataset")

print("Path to dataset files:", path)



import os

import kagglehub

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow.keras import layers, models

path = kagglehub.dataset_download("smaranjitghose/corn-or-maize-leaf-disease-dataset")

BATCH_SIZE = 32

IMG_SIZE = (128, 128)



adjusted_path = os.path.join(path, "data")

if not os.path.exists(adjusted_path):
```

```
    adjusted_path = path
train_ds = tf.keras.utils.image_dataset_from_directory(
    adjusted_path,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
)
val_ds = tf.keras.utils.image_dataset_from_directory(
    adjusted_path,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
)

class_names = train_ds.class_names
num_classes = len(class_names)
print(f"Detected Corn/Maize Disease classes: {class_names}")
plt.figure(figsize=(10, 10))
for images, labels in train_ds.take(1):
  for i in range(9):
    ax = plt.subplot(3, 3, i + 1)
    plt.imshow(images[i].numpy().astype("uint8"))
```

```
    plt.title(class_names[labels[i]])
    plt.axis("off")
plt.show()
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)


model = models.Sequential(
    [
        layers.Rescaling(1.0 / 255, input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3)),
        layers.Conv2D(32, (3, 3), activation="relu"),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(64, (3, 3), activation="relu"),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(128, (3, 3), activation="relu"),
        layers.MaxPooling2D((2, 2)),
        layers.Flatten(),
        layers.Dense(128, activation="relu"),
        layers.Dropout(0.5),
        layers.Dense(num_classes, activation="softmax"),
    ]
)
model.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=0.0001
    ),
```

```
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
model.summary()


EPOCHS = 10
history = model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS)
acc = history.history["accuracy"]
val_acc = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs_range = range(EPOCHS)
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label="Training Accuracy")
plt.plot(epochs_range, val_acc, label="Validation Accuracy")
plt.legend(loc="lower right")
plt.title("Training and Validation Accuracy")
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label="Training Loss")
plt.plot(epochs_range, val_loss, label="Validation Loss")
plt.legend(loc="upper right")
plt.title("Training and Validation Loss")
plt.show()


import numpy as np
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

y_true = []
y_pred = []
for images, labels in val_ds:
    y_true.extend(labels.numpy())
    predictions = model.predict(images)
    y_pred.extend(np.argmax(predictions, axis=1))
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for Corn/Maize Leaf Disease Classification')
plt.show()

from sklearn.metrics import classification_report
report = classification_report(y_true, y_pred, target_names=class_names)
print("Classification Report:\n", report)
```

Output :

Visualisation of data:

1. Model Summaey

Model: "sequential_6"

Layer (type)	Output Shape	Param #
rescaling_6 (Rescaling)	(None, 128, 128, 3)	0
conv2d_18 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_18 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_19 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_19 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_20 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_20 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_6 (Flatten)	(None, 25088)	0
dense_12 (Dense)	(None, 128)	3,211,392
dropout_4 (Dropout)	(None, 128)	0
dense_13 (Dense)	(None, 4)	516

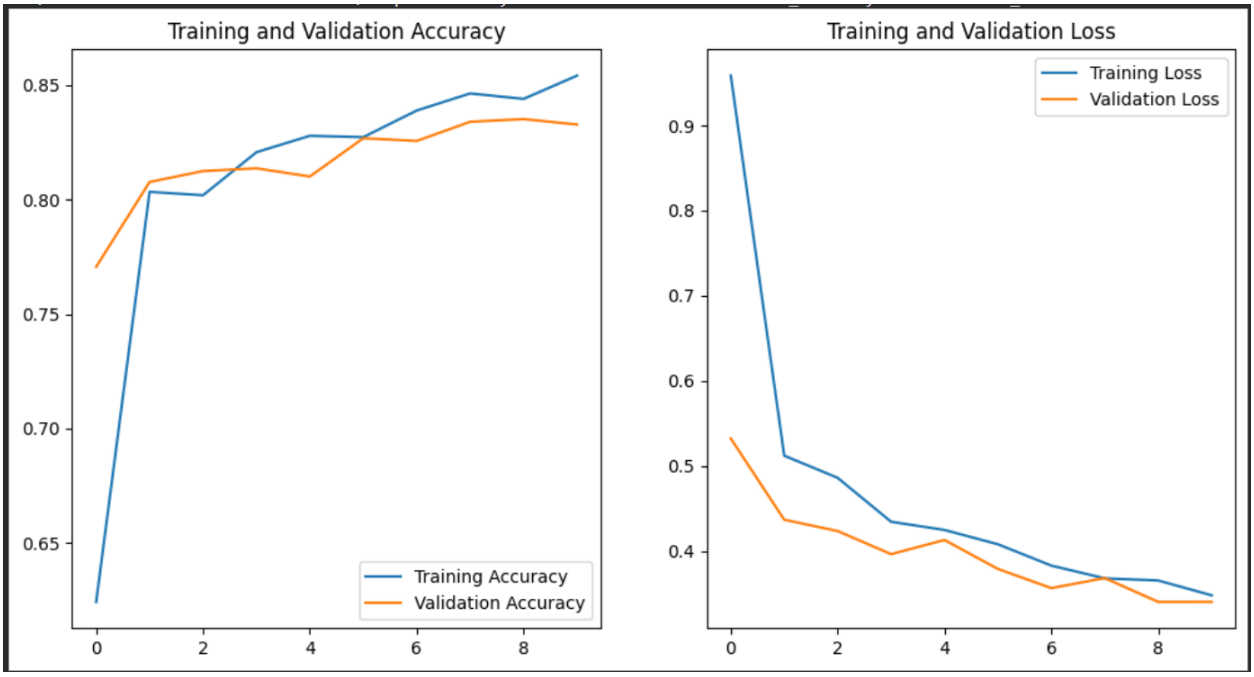
Total params: 3,305,156 (12.61 MB)
Trainable params: 3,305,156 (12.61 MB)
Non-trainable params: 0 (0.00 B)

2. Classify and print different classes

```
Using Colab cache for faster access to the 'corn-or-maize-leaf-disease-dataset' dataset.
Found 4188 files belonging to 4 classes.
Using 3351 files for training.
Found 4188 files belonging to 4 classes.
Using 837 files for validation.
Detected Corn/Maize Disease classes: ['Blight', 'Common_Rust', 'Gray_Leaf_Spot', 'Healthy']
```

3. Effect on Accuracy and loss of training and testing data as no. of epoch increases

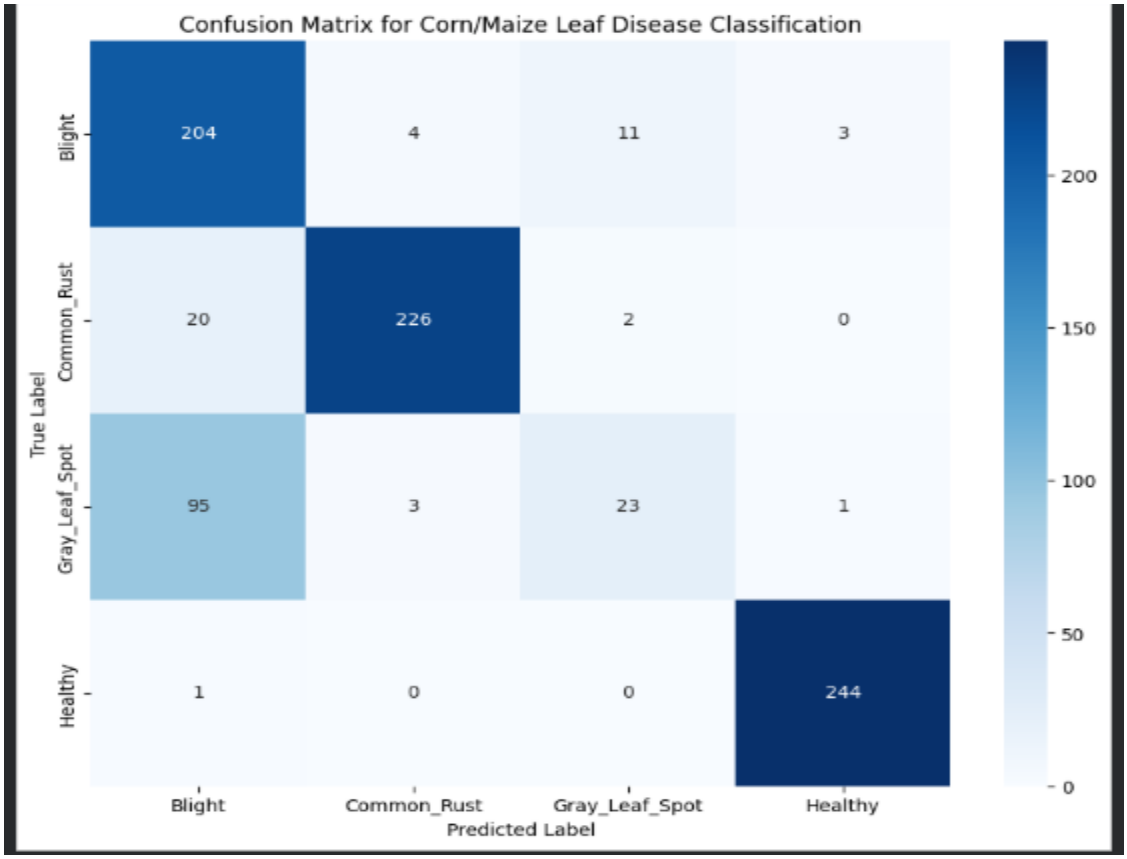
```
Epoch 1/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 15s 52ms/step - accuracy: 0.6243 - loss: 0.9584 - val_accuracy: 0.7706 - val_loss: 0.5322
Epoch 2/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 21ms/step - accuracy: 0.8033 - loss: 0.5121 - val_accuracy: 0.8076 - val_loss: 0.4369
Epoch 3/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.8019 - loss: 0.4861 - val_accuracy: 0.8124 - val_loss: 0.4236
Epoch 4/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.8207 - loss: 0.4345 - val_accuracy: 0.8136 - val_loss: 0.3965
Epoch 5/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.8278 - loss: 0.4249 - val_accuracy: 0.8100 - val_loss: 0.4131
Epoch 6/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.8272 - loss: 0.4081 - val_accuracy: 0.8268 - val_loss: 0.3791
Epoch 7/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.8389 - loss: 0.3830 - val_accuracy: 0.8256 - val_loss: 0.3566
Epoch 8/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - accuracy: 0.8463 - loss: 0.3680 - val_accuracy: 0.8339 - val_loss: 0.3684
Epoch 9/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.8439 - loss: 0.3655 - val_accuracy: 0.8351 - val_loss: 0.3404
Epoch 10/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 0.8541 - loss: 0.3481 - val_accuracy: 0.8327 - val_loss: 0.3404
```



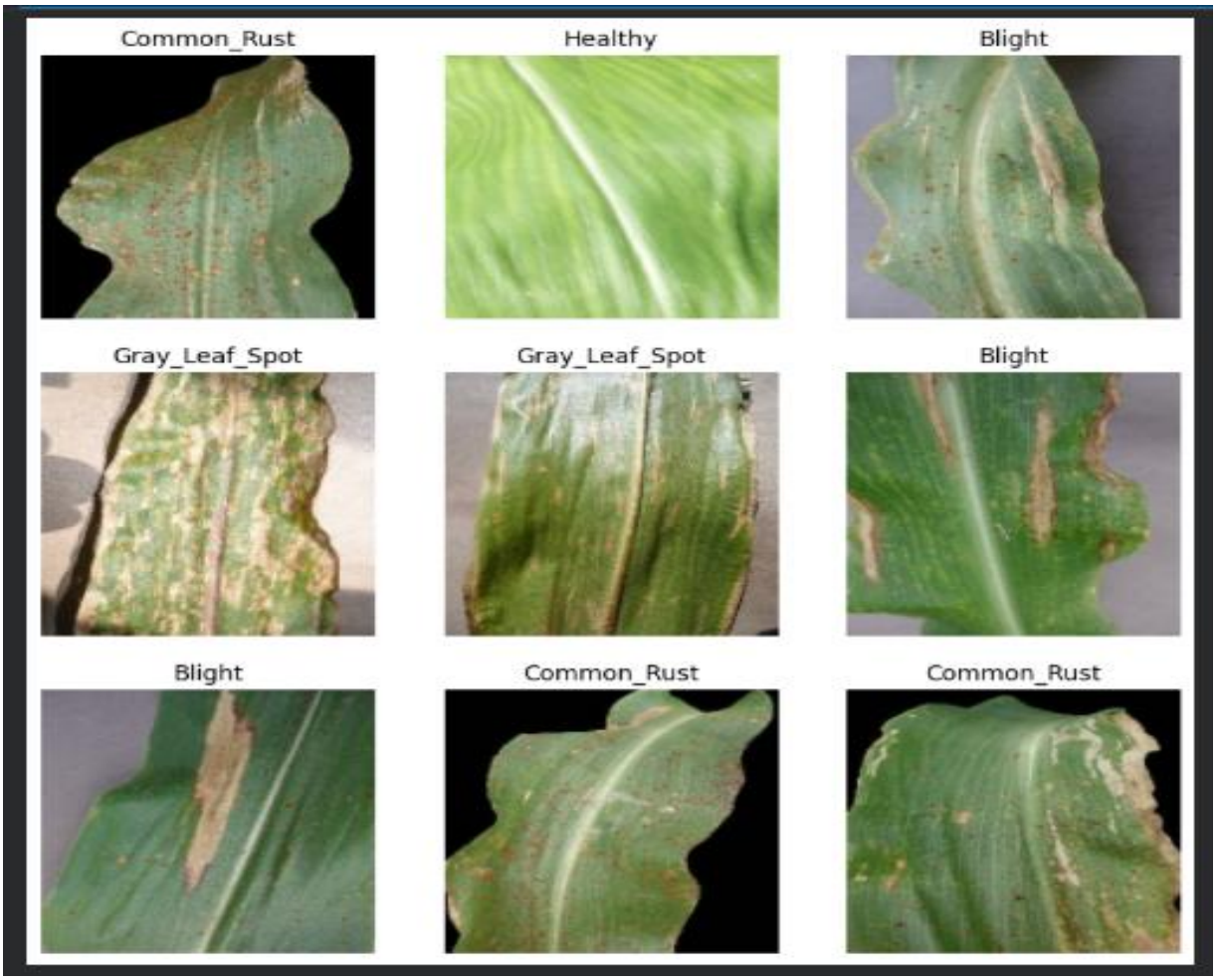
4. Classification Report and confusion matrix

Classification Report:

	precision	recall	f1-score	support
Blight	0.64	0.92	0.75	222
Common_Rust	0.97	0.91	0.94	248
Gray_Leaf_Spot	0.64	0.19	0.29	122
Healthy	0.98	1.00	0.99	245
accuracy			0.83	837
macro avg	0.81	0.75	0.74	837
weighted avg	0.84	0.83	0.81	837



5. Visualizing dataset


Common_Rust
Healthy
Blight
Gray_Leaf_Spot
Gray_Leaf_Spot
Blight
Blight
Common_Rust
Common_Rust